Adding a Tip Jar to a Webpage

August 15, 2026 · 13 min · 2624 words

▸ Table of Contents

The previous tutorials drove the `TipJar` contract entirely from the shell. This one puts it behind a webpage: a Connect button, a tip form, a live balance, and a withdraw button that only appears for the owner.

The browser flow is:

```
Webpage → ethers.js → MetaMask → Anvil → TipJar contract
```

MetaMask holds the keys and signs the transactions. **No private key goes anywhere near the JavaScript.** Frontend source is visible to anyone who visits the page, so a key in it is a key you have given away.

It follows on from Writing a Smart Contract with Foundry, which builds and deploys the contract used here. Its sibling is Invariant Testing with Foundry, which goes the other direction and tests the contract harder.

What You Will Use

- `anvil` — the local blockchain, as before
- `forge` — to deploy, and to generate the ABI
- **MetaMask** — a browser extension that holds keys and signs transactions
- **ethers.js** — the library your page uses to talk to MetaMask
- **Vite** — a dev server, so the page is served over HTTP rather than `file://`

Prerequisites

This tutorial differs from the others in one important way: **the browser runs on the host, not in the container.** MetaMask is a browser extension, so it must be able to reach both the dev server and Anvil's JSON-RPC endpoint from outside the container.

That means two ports have to be published, which the container from the earlier tutorials does not do. If you already have one, replace it:

`host : ~ : main`

```
podman rm -f foundry
```

Then create it with the ports exposed:

`host : ~ : main`

```
mkdir -p ~/tip-jar-work

podman run -dit \
    --name foundry \
    --volume ~/tip-jar-work:/work:Z \
    --workdir /work \
    --publish 8545:8545 \
    --publish 5173:5173 \
    ubuntu sleep infinity

podman exec -it foundry bash
```

```
8545   Anvil's JSON-RPC, so MetaMask can reach the chain
5173   Vite's dev server, so the browser can load the page
```

Install Foundry and Node. The base image has neither, and this is the first tutorial in the series that needs `npm`:

`container : /work : main`

```
export DEBIAN_FRONTEND=noninteractive
export TZ=Etc/UTC

apt-get update

apt-get install -y curl git ca-certificates jq nodejs npm

curl -L https://foundry.paradigm.xyz | bash
echo 'export PATH="$PATH:/root/.foundry/bin"' >> ~/.bashrc
source ~/.bashrc
foundryup

forge --version
node --version
```

Setup an empty project

```
cd /work
forge init tip-jar --empty
cd tip-jar
```

You also need the `TipJar` project itself. If you do not already have it, create it and paste in the contract from Section 2 of the previous tutorial:

`container : /work : main`

Finally, set the environment variables. These are Anvil's published development keys, identical on every machine:

`container : /work/tip-jar : main`

```
export RPC_URL=http://127.0.0.1:8545

export OWNER_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
export TIPPER_KEY=0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d

export OWNER=$(cast wallet address --private-key "$OWNER_KEY")
export TIPPER=$(cast wallet address --private-key "$TIPPER_KEY")
```

Where Each Command Runs

Every command block is preceded by a line saying where it belongs:

```
`container : /work/tip-jar : main`
     |            |            |
     |            |            └── which window
     |            └── the working directory
     └── host machine, or inside the container
```

This tutorial uses three shells and a browser:

```
main      the shell you work in
anvil     the shell running the local blockchain
web       the shell running the Vite dev server

browser   on the host, with MetaMask installed
```

---

1. Start Anvil

In a second shell, and note the extra flag:

`host → container : /work/tip-jar : anvil`

```
podman exec -it foundry bash
cd /work/tip-jar

anvil --host 0.0.0.0
```

By default Anvil binds to `127.0.0.1`, which inside a container means “reachable only from inside this container”. `--host 0.0.0.0` makes it listen on all interfaces so the published port actually forwards, and MetaMask on the host can reach it.

Anvil is now available at:

```
http://127.0.0.1:8545        from inside the container
http://127.0.0.1:8545        from the host, via --publish
```

with chain ID:

```
31337
```

Leave it running.

---

2. Deploy the TipJar Contract

Back in the main shell:

`container : /work/tip-jar : main`

```
forge build

forge create src/TipJar.sol:TipJar \
    --private-key "$OWNER_KEY" \
    --rpc-url "$RPC_URL" \
    --broadcast
```

You should get output similar to:

```
Deployer:         0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
Deployed to:      0x...
Transaction hash: 0x...
```

Save the address printed after `Deployed to:`, because the frontend needs it too:

`container : /work/tip-jar : main`

```
export TIPJAR=0xYOUR_CONTRACT_ADDRESS
```

Deploying with `$OWNER_KEY` is what makes Anvil account `(0)` the owner, since the constructor records `msg.sender`. That matters in **Section 11**, where only that account can withdraw.

Anvil resets when it is restarted. If you restart it, deploy again and update the address in the frontend as well as in `$TIPJAR`.

---

3. Set up MetaMask

https://support.metamask.io/start/getting-started-with-metamask/

3. Add Anvil to MetaMask

In the browser, add a custom network:

```
Network name: Anvil
RPC URL:      http://127.0.0.1:8545
Chain ID:     31337
Currency:     ETH
```

You can also let the JavaScript in **Section 7** ask MetaMask to add the network automatically, which is what `wallet_addEthereumChain` does.

---

4. Import Test Accounts into MetaMask

1. Add wallet
2. Via a private key

For sending tips, import Anvil account `(1)`:

```
0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d
```

For testing owner withdrawals, import account `(0)`:

```
0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
```

These are Anvil's well-known development accounts, and they are the same `$TIPPER_KEY` and `$OWNER_KEY` the shell has been using.

Never use these keys for real funds, for a public testnet account you care about, or on mainnet. They are published in Anvil's documentation; anyone can sweep them. Consider using a separate browser profile for local development so they never sit alongside a real wallet.

---

5. Set Up the Frontend

Create a Vite project alongside the contract, and add ethers:

`container : /work/tip-jar : main`

```
npm_config_yes=true npm create vite@latest frontend -- --template vanilla --no-interactive --no-immediate
cd frontend
npm install --no-audit --no-fund --progress=false
npm install ethers --no-audit --no-fund --progress=false
```

The parts that matter:

```
frontend/
├── index.html
├── package.json
└── src/
    └── tipjar.js
```

You can delete the rest of the scaffolding Vite generates.

---

6. Add the HTML

Replace the contents of `frontend/index.html`'s `<body>` with:

```
<section>
  <h1>Tip Jar</h1>

  <button id="connect">Connect wallet</button>

  <p>
    Wallet:
    <span id="wallet">Not connected</span>
  </p>

  <p>
    Contract balance:
    <strong><span id="balance">0</span> ETH</strong>
  </p>

  <p>
    Contract owner:
    <span id="owner">Unknown</span>
  </p>

  <div>
    <label for="amount">Tip amount</label>
    <input
      id="amount"
      type="number"
      min="0"
      step="0.01"
      value="0.1"
    />
  </div>

  <div>
    <label for="message">Message</label>
    <input
      id="message"
      type="text"
      value="Great work!"
    />
  </div>

  <button id="tip" disabled>Send tip</button>
  <button id="withdraw" disabled>Withdraw</button>
  <button id="refresh" disabled>Refresh balance</button>

  <pre id="status"></pre>
</section>

<script type="module" src="/src/tipjar.js"></script>
```

---

7. Create `src/tipjar.js`

Replace `PASTE_CONTRACT_ADDRESS_HERE` with the address `forge create` printed in **Section 2** — the same value as `$TIPJAR`.

```
import {
  BrowserProvider,
  Contract,
  formatEther,
  parseEther,
} from "ethers";

const CONTRACT_ADDRESS = "PASTE_CONTRACT_ADDRESS_HERE";

const ANVIL_CHAIN_ID = "0x7a69"; // 31337

const TIP_JAR_ABI = [
  "function owner() view returns (address)",
  "function getBalance() view returns (uint256)",
  "function tipsByAddress(address) view returns (uint256)",
  "function tip(string message) payable",
  "function withdraw()",
  "event TipReceived(address indexed sender, uint256 amount, string message)",
  "event Withdrawn(address indexed owner, uint256 amount)",
];

const connectButton = document.querySelector("#connect");
const tipButton = document.querySelector("#tip");
const withdrawButton = document.querySelector("#withdraw");
const refreshButton = document.querySelector("#refresh");

const walletElement = document.querySelector("#wallet");
const ownerElement = document.querySelector("#owner");
const balanceElement = document.querySelector("#balance");
const amountInput = document.querySelector("#amount");
const messageInput = document.querySelector("#message");
const statusElement = document.querySelector("#status");

let provider;
let signer;
let contract;
let connectedAddress;

function setStatus(message) {
  statusElement.textContent = message;
}

function getErrorMessage(error) {
  return (
    error?.shortMessage ||
    error?.reason ||
    error?.info?.error?.message ||
    error?.message ||
    String(error)
  );
}

async function selectAnvilNetwork() {
  try {
    await window.ethereum.request({
      method: "wallet_switchEthereumChain",
      params: [{ chainId: ANVIL_CHAIN_ID }],
    });
  } catch (error) {
    if (error.code !== 4902) {
      throw error;
    }

    await window.ethereum.request({
      method: "wallet_addEthereumChain",
      params: [
        {
          chainId: ANVIL_CHAIN_ID,
          chainName: "Anvil",
          nativeCurrency: {
            name: "Ether",
            symbol: "ETH",
            decimals: 18,
          },
          rpcUrls: ["http://127.0.0.1:8545"],
```

```
        },
      ],
    });
  }
}

async function connectWallet() {
  try {
    if (!window.ethereum) {
      throw new Error("MetaMask is not installed.");
    }

    setStatus("Connecting to MetaMask...");

    await window.ethereum.request({
      method: "eth_requestAccounts",
    });

    await selectAnvilNetwork();

    provider = new BrowserProvider(window.ethereum);
    signer = await provider.getSigner();
    connectedAddress = await signer.getAddress();

    contract = new Contract(
      CONTRACT_ADDRESS,
      TIP_JAR_ABI,
      signer,
    );

    walletElement.textContent = connectedAddress;
    connectButton.textContent = "Wallet connected";

    tipButton.disabled = false;
    refreshButton.disabled = false;

    await refreshContract();

    setStatus("Connected to Anvil.");
  } catch (error) {
    console.error(error);
    setStatus(`Connection failed: ${getErrorMessage(error)}`);
  }
}

async function refreshContract() {
  if (!contract) {
    return;
  }

  try {
    const [balance, owner] = await Promise.all([
      contract.getBalance(),
      contract.owner(),
    ]);

    balanceElement.textContent = formatEther(balance);
    ownerElement.textContent = owner;

    withdrawButton.disabled =
      owner.toLowerCase() !== connectedAddress.toLowerCase();
  } catch (error) {
    console.error(error);
    setStatus(
      `Could not read contract: ${getErrorMessage(error)}. ` +
      "Check that Anvil is running and the contract address is correct.",
    );
  }
}

async function sendTip() {
  try {
    const amount = amountInput.value.trim();
    const message = messageInput.value.trim();
```

```
  if (!amount || Number(amount) <= 0) {
    throw new Error("Enter a tip greater than zero.");
  }

  if (!message) {
    throw new Error("Enter a message.");
  }

  tipButton.disabled = true;
  setStatus("Confirm the tip in MetaMask...");

  const transaction = await contract.tip(message, {
    value: parseEther(amount),
  });

  setStatus(`Transaction submitted:\n${transaction.hash}`);

  await transaction.wait();

  setStatus(`Tip confirmed:\n${transaction.hash}`);
  await refreshContract();
} catch (error) {
  console.error(error);
  setStatus(`Tip failed: ${getErrorMessage(error)}`);
} finally {
  tipButton.disabled = false;
  }
}

async function withdraw() {
  try {
    withdrawButton.disabled = true;
    setStatus("Confirm the withdrawal in MetaMask...");

    const transaction = await contract.withdraw();

    setStatus(`Withdrawal submitted:\n${transaction.hash}`);

    await transaction.wait();

    setStatus(`Withdrawal confirmed:\n${transaction.hash}`);
    await refreshContract();
  } catch (error) {
    console.error(error);
    setStatus(`Withdrawal failed: ${getErrorMessage(error)}`);
  } finally {
    await refreshContract();
  }
}

connectButton.addEventListener("click", connectWallet);
tipButton.addEventListener("click", sendTip);
withdrawButton.addEventListener("click", withdraw);
refreshButton.addEventListener("click", refreshContract);

window.ethereum?.on("accountsChanged", () => {
  window.location.reload();
});

window.ethereum?.on("chainChanged", () => {
  window.location.reload();
});
```

There is a lot here, but it is four ideas:

```
BrowserProvider   wraps window.ethereum, the object MetaMask injects
signer            an account that can sign; comes from the provider
Contract          address + ABI + signer, giving you contract.tip(...)
parseEther        the same ETH-to-wei conversion cast to-wei does
```

The ABI is the bridge. It is the same information `cast` needs when you type `"tip(string)"` on the command line, just declared up front so ethers can encode calls for you.

Two details worth noticing. `refreshContract` enables the withdraw button only when the connected address

matches `owner()`, which is a courtesy, not security — the contract's own `require` is what actually enforces it. And the `accountsChanged` and `chainChanged` handlers reload the page, because a signer bound to the old account would otherwise keep signing as the wrong person.

---

8. Run the Frontend

`host → container : /work/tip-jar/frontend : web`

```
podman exec -it foundry bash
cd tip-jar/frontend

npm run dev -- --host 0.0.0.0
```

The `--host 0.0.0.0` is needed for the same reason Anvil needed it: Vite binds to localhost by default, which inside a container is unreachable from the host.

Vite will print something similar to:

```
http://localhost:5173
```

Open that in a browser with MetaMask installed.

---

9. Send a Tip

On the webpage:

1. Click **Connect wallet**.
2. Connect Anvil account `(1)`, the tipper.
3. Make sure MetaMask is on the `Anvil` network.
4. Enter `0.1` as the tip amount.
5. Enter a message, for example `Great work!`.
6. Click **Send tip**.
7. Approve the transaction in MetaMask.

Once mined, the page should show:

```
Contract balance: 0.1 ETH
```

Watch the Anvil window while you do this. It logs the `eth_sendRawTransaction` and mines a block, exactly as it did for `cast send`. The browser is doing nothing the command line was not.

---

10. Verify from the Command Line

The webpage says the tip arrived. Confirm it independently, which is the whole advantage of having both interfaces:

`container : /work/tip-jar : main`

```
cast balance "$TIPJAR" \
    --ether \
    --rpc-url "$RPC_URL"
```

```
0.100000000000000000
```

And through the contract's own function:

`container : /work/tip-jar : main`

```
cast call "$TIPJAR" \
    "getBalance()(uint256)" \
    --rpc-url "$RPC_URL"
```

The message you typed into the form is in the event log, not in storage, and can be read back exactly as Section 17 of the previous tutorial describes:

`container : /work/tip-jar : main`

```
cast logs \
    "TipReceived(address indexed sender, uint256 amount, string message)" \
    --from-block 0 \
    --rpc-url "$RPC_URL"
```

---

11. Withdraw from the Webpage

Switch MetaMask to Anvil account `(0)` — the account that deployed the contract, and therefore its owner.

Reload the page, or reconnect MetaMask. The **Withdraw** button becomes enabled.

Click it and approve the transaction. The contract balance returns to:

```
0 ETH
```

Try it the other way round to see the contract defend itself: switch back to account `(1)`, and the button greys out. That is only the UI being polite. The real check is `require(msg.sender == owner, ...)` inside `withdraw()`, and a crafted call from account `(1)` would still revert — which is exactly what happened on the command line in Section 14 of the previous tutorial.

---

12. Using Foundry's Generated ABI

The example above declares the ABI by hand:

```
const TIP_JAR_ABI = [
  "function owner() view returns (address)",
  "function getBalance() view returns (uint256)",
  "function tipsByAddress(address) view returns (uint256)",
  "function tip(string message) payable",
  "function withdraw()",
];
```

That is fine for five functions and hopeless for fifty. Worse, it drifts: change the Solidity and the JavaScript silently keeps the old signature.

Foundry already writes a complete ABI during `forge build`:

```
out/TipJar.sol/TipJar.json
```

Extract just the ABI array from it:

`container : /work/tip-jar : main`

```
jq '.abi' \
    out/TipJar.sol/TipJar.json \
    > frontend/src/TipJar.abi.json
```

Then import it instead:

```
import TipJarABI from "./TipJar.abi.json";

contract = new Contract(
  CONTRACT_ADDRESS,
  TipJarABI,
  signer,
);
```

Re-run that `jq` line after any contract change and the frontend cannot fall out of sync. In a real project it belongs in a build script.

---

13. The Whole Development Loop

Three shells, in order:

`host → container : /work/tip-jar : anvil`

```
podman exec -it foundry bash
cd tip-jar
anvil --host 0.0.0.0
```

`container : /work/tip-jar : main`

```
forge build

forge create src/TipJar.sol:TipJar \
    --private-key "$OWNER_KEY" \
    --rpc-url "$RPC_URL" \
    --broadcast

export TIPJAR=0xYOUR_CONTRACT_ADDRESS

jq '.abi' out/TipJar.sol/TipJar.json > frontend/src/TipJar.abi.json
```

`host → container : /work/tip-jar/frontend : web`

```
podman exec -it foundry bash
cd tip-jar/frontend
npm run dev -- --host 0.0.0.0
```

Which assembles into:

```
Solidity
   ↓
forge build
   ↓
forge create
   ↓
Anvil
   ↑
MetaMask
   ↑
ethers.js
   ↑
Webpage
```

Read it as two halves meeting at Anvil. Everything above the chain is your code being compiled and deployed; everything below it is the browser asking MetaMask to sign, so the transaction arrives the same way `cast send` did.

---

14. The Rule That Matters

For local development, Anvil's default keys on the command line are fine. They are public, and the ETH is fake.

For anything real, the model is:

```
Browser → MetaMask → Smart Contract
```

Never this:

```
Browser JavaScript → Private Key → Smart Contract
```

A private key in a webpage should be considered compromised the moment the page is served. This is not a matter of obfuscation or minification — the browser has to be able to read the code in order to run it, and so can anyone else. Signing is the wallet's job precisely so your application never has to hold the secret.

The same applies to any key you paste into a terminal on a machine you do not control, or commit to a repository. Assume anything that leaves your control is public.

---

15. Additional Resources

- ethers.js documentation — v6, which is what the code above uses. Note that v5 examples on the web will not work unchanged
- MetaMask developer docs — the provider API, including `wallet_addEthereumChain` and `wallet_switchEthereumChain`
- EIP-1193 — the standard behind `window.ethereum`, which every browser wallet implements
- viem — the main modern alternative to ethers, worth knowing about if you start a new project
- Vite — the dev server used here

The Rest of the Series

- Ethereum from the Command Line — Anvil, accounts, transactions, and reading the real chain
- Writing a Smart Contract with Foundry — the contract this page talks to
- Invariant Testing with Foundry — the sibling of this tutorial: properties that must hold after any sequence of calls

---

Summary

```
Webpage      your HTML and JavaScript
   ↓
ethers.js    encodes calls using the ABI
   ↓
MetaMask     holds the key, signs, submits
   ↓
Anvil        mines the transaction
   ↓
TipJar       runs, and updates its state
```

Nothing in that chain is new except the top two rows. The contract, the chain, and the transactions are the same ones the command-line tutorials produced — the browser is just another JSON-RPC client, and the only genuinely new idea is that the key now lives in a wallet instead of an environment variable.

NEXT »
Invariant Testing with Foundry

Public domain (CC0) · Powered by Hugo & PaperMod